Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосувань»

Практична робота №11

Локальні сховища даних: localStorage/IndexedDB

Виконав:
Студент II курсу
групи ІП-56
Гончар І.В.

Перевірив:
Ковтунець О.В.

2026

1. **Мета роботи:**

Навчитися зберігати дані застосунку між перезавантаженнями сторінки: спочатку найпростішим способом - через localStorage (JSON.stringify/parse), а потім перейти на просту схему IndexedDB з одним об'єктним сховищем і базовими CRUD-операціями (створення, читання, оновлення, видалення). Окрема увага - різниці між синхронним доступом (localStorage блокує потік виконання) і асинхронним (IndexedDB працює через транзакції та колбеки/Promise), а також одноразовій міграції наявних даних із localStorage в IndexedDB.

2. **Хід виконання роботи**

Крок 1-4. Модель даних та реалізація збереження в localStorage:

Для предметної області "Каталог настільних ігор" (Варіант 4) до масиву об'єктів було додано поле owned (статус «Зіграно» / «Не зіграно») та унікальний ідентифікатор id. Реалізовано функції saveToLocalStorage та loadFromLocalStorage з використанням JSON.stringify та JSON.parse (загорнутого у блок try/catch для обробки пошкоджених даних). У компоненті Vue налаштовано спостерігач (watch) з параметром deep: true, який автоматично зберігає масив ігор у localStorage після кожного кліку по кнопці зміни статусу.

Дані успішно зберігаються між перезавантаженнями сторінки, що підтверджується перевіркою у панелі DevTools (рис. 1).

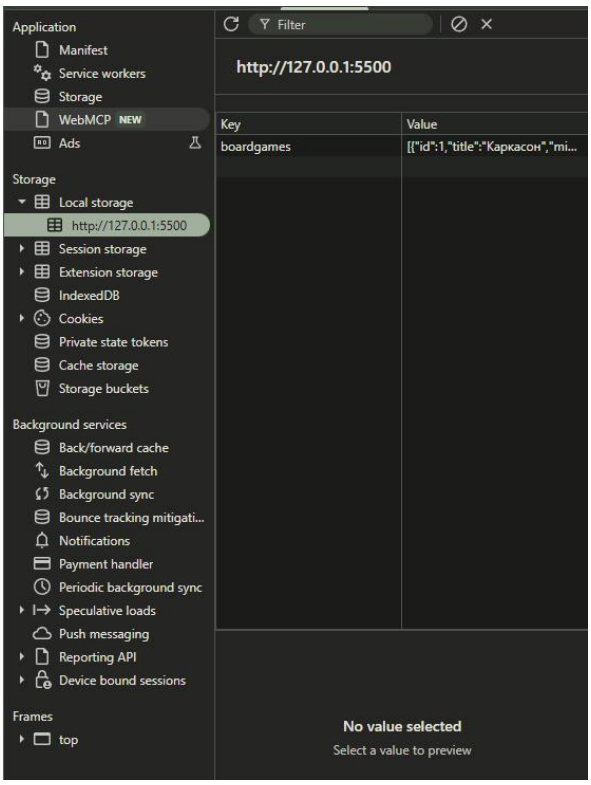


Рис. 1 — Перевірка збереження даних у localStorage

Крок 5-10. Проектування схеми IndexedDB, реалізація CRUD-операцій та міграція даних:

Для забезпечення асинхронного доступу та масштабованості, збереження даних було переведено на IndexedDB. Створено базу даних `BoardGamesDB` (версія 1), яка містить об'єктне сховище `games` із первинним ключем `keyPath: 'id'`. Зміна статусу «Зіграно» (оновлення поля `owned`) тепер записується в базу через транзакцію типу `readwrite` за допомогою методу `put`, що дозволяє оновлювати існуючий запис без створення дублікатів. Відображення списку ігор реалізовано через метод `getAll` у межах транзакції `readonly`.

Для збереження накопичених користувачем даних було реалізовано механізм одноразової міграції (`migrateDataIfNeeded`). При завантаженні сторінки скрипт перевіряє наявність даних у localStorage. Якщо вони існують і міграція ще не відбувалася, масив ігор асинхронно переноситься у сховище IndexedDB, після чого у localStorage встановлюється відповідний прапорець, щоб уникнути повторного копіювання в майбутньому.

Коректність роботи асинхронної бази даних, створення об'єктного сховища та успішність міграції записів було перевірено за допомогою вкладки IndexedDB у панелі інструментів розробника (рис. 2).

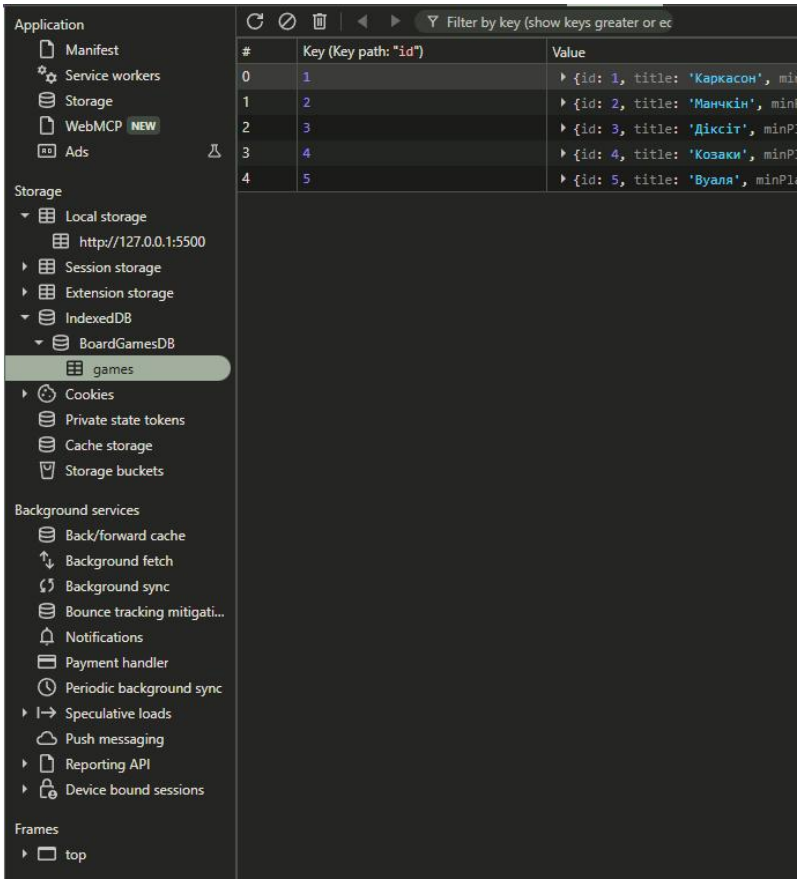


Рис. 2 — Перевірка структури та даних у сховищі IndexedDB

Крок 11-12. Версіонування та збереження:

Усі оновлення коду, що стосуються впровадження локальних баз даних (`localStorage` та `IndexedDB`), а також логіки міграції збережених записів, було успішно зафіксовано системою контролю версій Git. Зміни супроводжено описовим повідомленням та відправлено до віддаленого репозиторію на платформі GitHub (рис. 3).

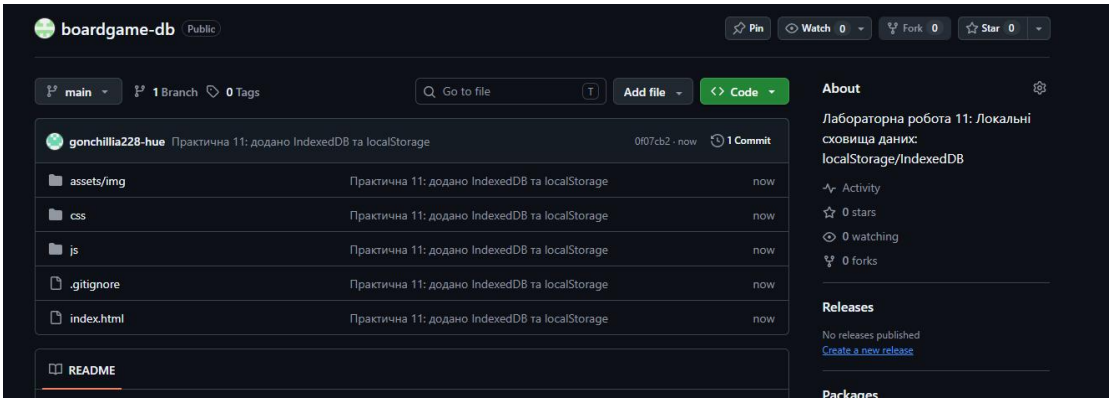


Рис. 3 — Фіксація змін у системі контролю версій Git на платформі GitHub

Посилання на репозиторій:

https://github.com/Bitshkiper218/boardgame-db

Висновок:

Під час виконання практичної роботи було успішно освоєно принципи роботи з локальними сховищами даних у браузері. На першому етапі було реалізовано синхронне збереження стану застосунку (статус «Зіграно» для каталогу настільних ігор) за допомогою `localStorage` та методів `JSON.stringify` і `JSON.parse`. На другому етапі застосунок було переведено на використання повноцінної асинхронної бази даних `IndexedDB`. Було створено базу `BoardGamesDB` з об'єктним сховищем `games`, де реалізовано CRUD-операції в межах транзакцій (зокрема метод `put` для оновлення статусу без створення дублікатів).

Для збереження накопиченого користувацького досвіду було успішно розроблено та налаштовано механізм одноразової міграції даних із `localStorage` до `IndexedDB`. Коректність роботи обох сховищ, збереження даних після перезавантаження сторінки та успішність міграції записів було перевірено за допомогою панелі інструментів розробника (DevTools). Фінальний код проєкту успішно версіоновано та збережено у віддаленому репозиторію на платформі GitHub.